

ECE 402: Communications Engineering Project I — Digitizing Audio Signals

Professor Brian L. Hughes
Department of Electrical and Computer Engineering
North Carolina State University
Raleigh, NC 27695-7911

©B. L. Hughes, 2025

Fall 2025

Contents

1	Working with MATLAB	3
1.1	Getting Started	3
1.2	Functions and Graphics	7
1.3	M-files and Scripts	9
1.4	Flow Control	11
1.5	Audio Files	13
2	Digital Signal Processing	18
2.1	The Discrete Fourier Transform	18
2.2	Relationship to Other Transforms	19
3	Sampling and Quantization	29
3.1	Sampling Tones	29
3.2	Uniform Quantization	30
3.3	Companding	33

Chapter 1

Working with MATLAB

1.1 Getting Started

MATLAB (short for MATrix LABoratory) is a powerful interactive tool for doing calculations with matrices and vectors. It can perform many common signal processing and communication tasks and display the results graphically. You can access MATLAB from any EOS lab computer. MATLAB is also available at no cost to all NCSU students to install on personal computers. To install, click on the link to the NCSU Mathworks portal, posted on the course website. In this section, we review some of the basic features of MATLAB used in this course. The best way to learn is to work through the examples at your computer.

On any campus computer, double-click the **MATLAB** icon to launch the graphical user interface. You can now enter commands in the *command window* – the window in the center that displays the prompt `>>`. Extensive tutorial and reference material for MATLAB are available online by typing

```
>> doc
```

in the command window, which opens MATLAB documentation. Open the MATLAB tutorial by clicking on “MATLAB” in the top left of the documentation window, and then clicking on the tutorial *Getting Started*. Before proceeding further, read all of the sections in this tutorial and work through the examples in the command window. *You are responsible for the material in this tutorial.*

The features of MATLAB are best illustrated with a sample session. In the examples below, the normal text is what you type, the bold text is the computer’s response, and the text on the right is an explanation (don’t type this). Follow along with these examples at the computer.

In MATLAB, all data are represented by matrices, including vectors ($1 \times n$ matrices) and scalars (1×1 matrices). Type the following in the command window:

```
>> a = 4                                Enter a scalar a
a =
    4
>> x = [1 -1 2 3]                       Enter a row vector x
x =
    1   -1    2    3
>> y = [0;1;-2;1]                       Enter a column vector y
y =
    0
    1
   -2
    1
>> z= y*x                               Multiply x and y to form a matrix z
z =
    0    0    0    0
    0   -1    2    3
   -2    2   -4   -6
    1   -1    2    3
```

For both column and row vectors, we can enter the elements in one line, where “;” marks the end of a row. Thus, we could have entered the matrix z directly by typing

```
>> z= [0 0 0 0; 0 -1 2 3; -2 2 -4 -6; 1 -1 2 3]   Enter the matrix z
z =
    0    0    0    0
    0   -1    2    3
   -2    2   -4   -6
    1   -1    2    3
```

The variables a , x , y and z are now stored in the **workspace**. To see the contents of the workspace, type

```
>> who                                     What variables are in the workspace?
Your variables are:
a    x    y    z
```

To determine the value of a variable, simply type it at the command prompt:

```
>> x
x =
```

```
1 -1 2 3
```

What is x ?

Note MATLAB echos everything you type. We can suppress this by adding a “;” to the end of the command

```
>> b = -6;
```

```
>> b
b =
```

```
-6
```

Enter b , suppressing echo

What is b ?

MATLAB will remember all variables until they are overwritten or erased (or until you exit the program). To erase a variable, type

```
>> clear z
```

Delete z from the workspace

The command

```
>> clear all
```

will delete *everything* from the workspace — use it carefully!

MATLAB uses the following arithmetic operators:

- + addition
- subtraction
- * multiplication
- / division
- ^ power operator
- ' conjugate-transpose
- ' transpose

For example to compute the Euclidean length of a real vector, $\|u\| = \sqrt{uu'}$, type

```
>> u = [1 2 3 4];
```

```
>> sqrt(u*u')
```

```
ans =
```

```
5.4772
```

Enter a vector u

Calculate the length

Note the MATLAB command `length(u)` returns the number of elements in a vector, NOT the Euclidean length. More generally, `[rows, columns]=size(u)` returns the number of rows and columns in `u`.

```
>> length(u)                                How many elements are in the vector u?
ans =
     4
```

The operations of addition and subtraction operate on an element-by-element basis, but the others are matrix operations. In MATLAB, a dot before `*` will cause it to perform entry-by-entry multiplication instead of the usual matrix operation. For example

```
>> [1 2 3 4].*[1 2 3 4]
ans =
     1     4     9    16
```

The same is true of `^` and `/`. Thus, the same result can be obtained with

```
>> [1 2 3 4].^2
ans =
     1     4     9    16
```

Element-by-element operations are particularly useful for plotting functions.

When you exit MATLAB, *all of your work is usually lost*. To save your work, you must first (a) set the MATLAB current directory to the directory where you want to save your work, and (b) save your work to a file in your directory. To change the MATLAB current directory, click the “browse for folder” icon above and to the left of the command window and navigate to your personal directory. To save your work in a file called “work.mat” in your personal directory, type

```
>> save work
```

The next time you start MATLAB, you can reload your work by again setting the MATLAB current directory to your personal directory and then typing

```
>> load work
```

To exit MATLAB, close the MATLAB GUI by clicking on the x in the upper right, or type

```
>> quit
```

```
121 flops  
%
```

1.2 Functions and Graphics

MATLAB has many built-in functions that make it easy to generate complex signals, manipulate them mathematically, and display the results graphically. For example, the command `a:h:b` generates a row vector $(a, a+h, a+2h, \dots)$ of entries that start at a and increase in increments of h , up to and possibly including b :

```
>> x = 1:2:10           Define a vector from 1 to 10 in increments of 2  
x =  
    1    3    5    7    9
```

Some other useful matrices:

```
zeros(n,m)  n × m matrix of zeros  
ones(n,m)   n × m matrix of ones  
eye(n,m)    n × m identity matrix
```

MATLAB's built-in math functions can be used to generate a variety of signals. For example, we can generate two cycles of the sine wave

$$y(t) = \sin(2\pi t) ,$$

by typing

```
>> time = 0:0.01:2;      Generate time from 0 to 2 in increments of 0.01  
>> y=sin(2*pi*time);     Generate a 1 Hz sine wave at these times  
>> plot(time,y)          Plot the signal
```

MATLAB is doing some impressive things here: First, MATLAB understands what `sin(x)` means, because sine is a *built-in function*. Second, we did not substitute just one value of time into sine, but rather the entire *vector* of times. Most built-in functions like `sin(x)` are *vectorized*, which means they accept vectors as inputs and return vectors equal to the function evaluated at each element of the input vector. Third, MATLAB understands what `pi` means. Many common constants, like `pi`, are *built-in variables* in MATLAB.

The last command, `plot(time,y)`, will cause a window to pop-up displaying the figure. You can use the window menu at the top to label the plot (**Title**, **X Label**, **Y Label**, etc) as shown in Fig. 1.1. You can also label the plot in the command window by typing

<code>>> xlabel('time (sec)')</code>	Enter x-axis label
<code>>> ylabel('y(t)')</code>	Enter y-axis label
<code>>> title('Plot of y(t) = sin(2*pi*t)')</code>	Enter plot title

One of the most powerful features of MATLAB is the tremendous variety of built-in functions. Some of the most useful are

<code>sin()</code>	sine
<code>cos()</code>	cosine
<code>abs()</code>	magnitude of number (absolute value if real)
<code>angle()</code>	angle of complex number (radians)
<code>log()</code>	base- e (natural) logarithm
<code>log10()</code>	base-10 logarithm
<code>exp()</code>	exponential
<code>sum()</code>	sum the elements of matrix
<code>real()</code>	real part of complex number
<code>imag()</code>	imaginary part of complex number
<code>conj()</code>	complex conjugate
<code>sign()</code>	algebraic sign (signum)

MATLAB also has an extensive collection of built-in variables, including

<code>e</code>	e
<code>pi</code>	π
<code>j</code>	$j = \sqrt{-1}$
<code>eps</code>	floating point relative precision 2^{-52}

You can learn more about functions and variables from MATLAB's online help, which can be accessed by clicking "?" in the upper right of the GUI, or by typing `help` followed by the function/variable name, e.g.

<code>>> help plot</code>	Find help on plot
---------------------------------	-------------------

Step functions and pulses can be easily represented in MATLAB using relational operators. Relational operators are expressions that resemble equations that take on the value 1 if the equation is true and 0 if the equation is false. For example,

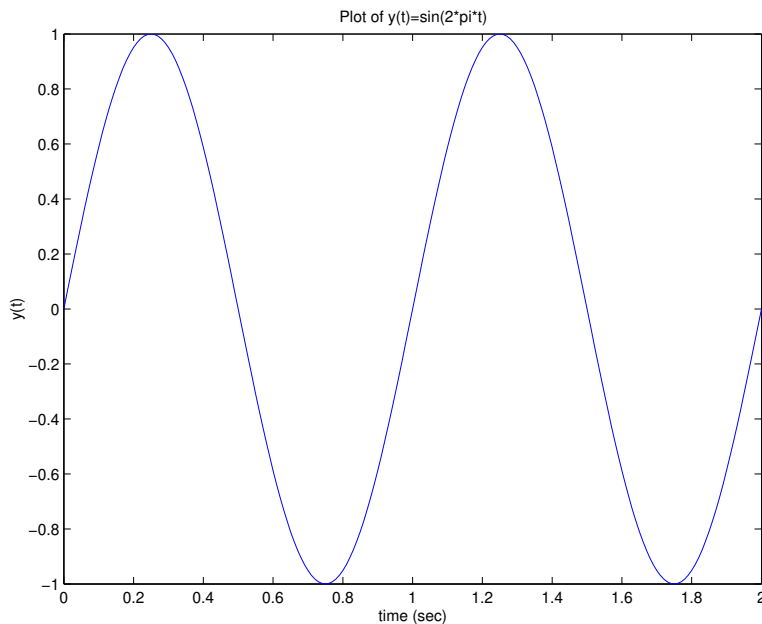


Figure 1.1: A simple plot using MATLAB.

Q1=(A>=B)

Is $A \geq B$?

Q2=(C==A+B)

Does C equal A+B?

Q3=(A<B)&(B<=C)

Is $A < B$ AND $B \leq C$?

Using these operators, we can represent the unit step function,

$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & \text{elsewhere} \end{cases}, \text{ by the expression } u = (t \geq 0),$$

where t is any vector of times. Here $(t \geq 0)$ takes on the value 1 for any $t \geq 0$ and takes on the value 0 otherwise. Similarly, we can represent the rectangle function

$$\Pi(t) = \begin{cases} 1, & -1/2 \leq t \leq 1/2 \\ 0, & \text{elsewhere} \end{cases}, \text{ by the expression } \text{rect} = (t \geq -0.5) - (t > 0.5).$$

1.3 M-files and Scripts

MATLAB also allows you to create your own functions using *m-files*. An m-file is a text file whose filename ends in `.m` (e.g., `cos.m`). You can create and edit m-files with MATLAB's

built-in visual editor, which can be launched by clicking the menu **New** at the far left of the toolbar above the command window and selecting **Script**. There are two kinds of m-files, *functions* and *scripts*.

Function m-files are used to define your own functions, which can be used in the same way as MATLAB's built-in functions, such as `cos` and `sin`. To define a new function called **function-name**, create a new m-file and name it *function-name.m*. The first line of the file must start with the word “function” and takes the form

```
function y=function-name(argument1, argument2,...)
```

which specifies the name of the function and its inputs. For example, suppose we want to define a new function called `rect(t)`, which calculates the rectangle function in the last section. To do so, use the MATLAB editor to create a file `rect.m` which contains the commands

```
function y=rect(t)
% rect(t): This function computes the rectangle function, which takes
% the value 1 for -0.5 <= t <= 0.5 and is zero everywhere else
y = (t>=-0.5)-(t>0.5);
```

You can now use `rect(t)` like any other function in MATLAB. The comments after the first line of the m-file provide help on using the function:

```
>> help rect                                Search for help on rect
rect(t): This function computes the rectangle function, which takes
the value 1 for -0.5 <= t <= 0.5 and is zero everywhere else
```

We can also use m-files to create MATLAB *scripts* — programs written in the MATLAB language. It is usually easier to perform a chain of calculations using scripts, rather than entering commands interactively. For example, in MATLAB, use the “browse for folder” button to navigate to your personal directory. Now use the MATLAB script editor to create an m-file called `sinplot.m` in your personal directory, which contains the commands

```
time = 0:0.01:2;
y=sin(2*pi*time);
plot(time,y)
xlabel('time (sec)')
ylabel('y(t)')
title('Plot of y(t) = sin(2*pi*t)')
```

Now run the script by clicking the “run” icon above the editor window, or by typing

```
>> sinplot
```

in the command window. This produces the same plot as if the commands were entered interactively.

1.4 Flow Control

MATLAB has several flow control statements, such as **for**, **if**, **while**, **continue**, and **break**, which are useful for writing more complex scripts. These statements work in much the same way as they do in the C programming language. In this section, we consider only the **for** statement; for information on the others, refer to the MATLAB documentation.

The **for** command allows you to execute a set of commands several times, just as it does in C. For example, the commands

<pre>>> for n=1:3</pre>	Begin loop
<pre>n</pre>	Type n
<pre>end</pre>	End loop
<pre> n =</pre>	
<pre> 1</pre>	
<pre> n =</pre>	
<pre> 2</pre>	
<pre> n =</pre>	
<pre> 3</pre>	

cause MATLAB to index n from 1 to 3, printing its value at each step. Every **for** statement must have a matching **end**, and the result is not echoed until the loop is closed.

The **for** command is very useful for summing series. For example, to calculate

$$x = \sum_{n=1}^{10} n^2,$$

we could use the script

<pre>x = 0;</pre>	%initial value of x
<pre>for n=1:10</pre>	%begin loop
<pre> x = x + n^2;</pre>	%add n-th term
<pre>end</pre>	
<pre>x</pre>	%print x

which produces the response

x =

385

We conclude this section with an example that exercises much of what we have learned so far. In this example, we use a `for` loop to approximate a square wave using its Fourier Series. In the notes, we show that the Fourier series for a general square wave $x(t)$ is given by:

$$x(t) = \sum_{n=-\infty}^{\infty} \Pi\left(\frac{t-nT}{\tau}\right) = \sum_{n=-\infty}^{\infty} d \operatorname{sinc}(nd) \exp(2\pi j n f_0 t) , \quad (1.1)$$

where τ is the pulse width, T is the period, $f_0 = 1/T$ is the fundamental frequency, and $d = \tau/T$ is the duty factor.

Consider the special case where $\tau = 1$, $T = 2$, and so $d = 1/2$. We want to plot one period of $x(t)$ together with a Fourier Series approximation using the terms $n = -15, -14, \dots, 15$. For the period $-1 \leq t \leq 1$, note that $x(t)$ has only one nonzero pulse which is equal to the rectangle function, $\Pi(t)$. As shown in the last section, we can calculate $\Pi(t)$ using the expression $(t \geq -0.5) - (t > 0.5)$, or the function `rect.m` that we created in the last section.

To plot $x(t)$ together with its approximation, use the text editor to create an m-file, say `p1fourier.m`, with the following contents:

```
% p1fourier.m - This script approximates a square wave with tau=1 and T=2,
% using only the terms in the Fourier Series between n=-15 and n=15
clear all; close all; %clear old values, close old plots

t=-1:0.01:1; %generate time from -1 to 1
T=2; d=0.5; f0=1/T; %period, duty cycle, fund frequency
x=(t>=-0.5)-(t>0.5); %generate the square wave x(t)

fs=0; %initialize FS to zero
for n=-15:15 %loop to add terms from -15 to +15
    fs=fs+d*sinc(n*d)*exp(2*pi*j*n*f0*t);
end
plot(t,x,t,real(fs)) %plot x(t) together with FS approx
axis([-1,1,-0.2,1.2]) %set min and max x and y values
xlabel('time (sec)')
ylabel('x(t)')
legend('x(t)', 'FS Approximation') %add legend to distinguish curves
title(['Fourier Series Approximation of Square Wave (n=-15 to 15)'])
```

This file is also posted under “Project Scripts” on the course website. Running this script in MATLAB will generate the plot in Fig. 1.2.

In theory, the Fourier Series (1.1) is always real whenever the corresponding signal is real. However, small numerical errors in MATLAB can cause this series to have a small imaginary

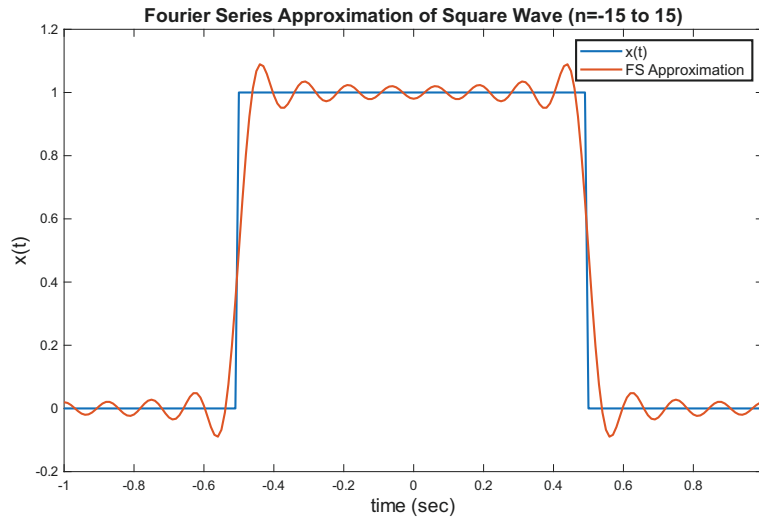


Figure 1.2: Fourier series approximation of square wave ($n = 15$).

part, which will cause an error message in MATLAB. There are two possible fixes for this problem: (a) we can convert the Fourier series to real form, or (b) we can use the command `real(fs)` to remove the imaginary part before plotting, as we did in the script above.

1.5 Audio Files

Note: The projects in this manual require MATLAB R2024b or later with the Communications, DSP System, and Signal Processing Toolboxes. To check that these toolboxes are installed, type `ver` at the MATLAB command prompt and check for “Communications Systems Toolbox,” “DSP System Toolbox,” and “Signal Processing Toolbox” in the list of installed products.

MATLAB can read, play, and write sound files in `wav` and `au` formats. In this course, we will deal mostly with the `wav` format, which is a form of pulse code modulation. To work the examples in this section, you will need a computer with a sound card, and speakers or headphones. You can use any `wav` file from the Internet for this demonstration, provided it is monaural. For example, you can download a `wav` file from the ECE 402 course “Projects” website. If you have a microphone, feel free to record your own voice.

Now suppose you have downloaded a file named `file.wav` to your personal directory. Start MATLAB, navigate to the directory that contains the file, and read it in as follows:

```
>> [x,fs]=audioread('file.wav');
```

 Read file, f_s

The variable `x` should now be a column vector that contains audio samples

$$x = [x(0), x(T), x(2T), \dots, x([N-1]T)]'$$

in the range $-1 < x(nT) < 1$, taken at a sampling rate of $f_s = 1/T$ Hertz. The duration of the clip is $(N-1)T$, where the time between samples is $T = 1/f_s$. You can find N by typing `N = length(x)`. If x has two columns, it is a stereo file. Obtain a monaural track by extracting the first column:

```
>> x = x(1:length(x),1);
```

 Extract L stereo track

For example, we can read and plot the

To play the file, type

```
>> sound(x,fs);
```

 Play the file

Now that the signal is in digital form, you can change it in many ways:

```
>> sound(2*x,fs);
```

 Amplify it

```
>> sound(x,2*fs);
```

 Play at twice normal speed

```
>> sound(flipud(x),fs);
```

 Play backwards

```
>> sound(x+flipud(x),fs);
```

 Play both together

We can even add an echo!

```
>> dead=zeros(fs,1);
```

 Create 1 s of silence

```
>> x1=[x;dead];
```

 Add 1 s of silence to `x`

```
>> x2=0.5*[dead;x];
```

 Create a smaller echo

```
>> sound(x1+x2,fs);
```

 Now play them together

Finally, you can save a modified `wav` file with the command

```
>> audiowrite('new.wav',x1,fs)
```

 Write a new `wav` file

We can also combine these commands with earlier results to plot the audio signal in time. For example, the following script reads and plots audio file `sound0.wav`, which is available on the course webpage:

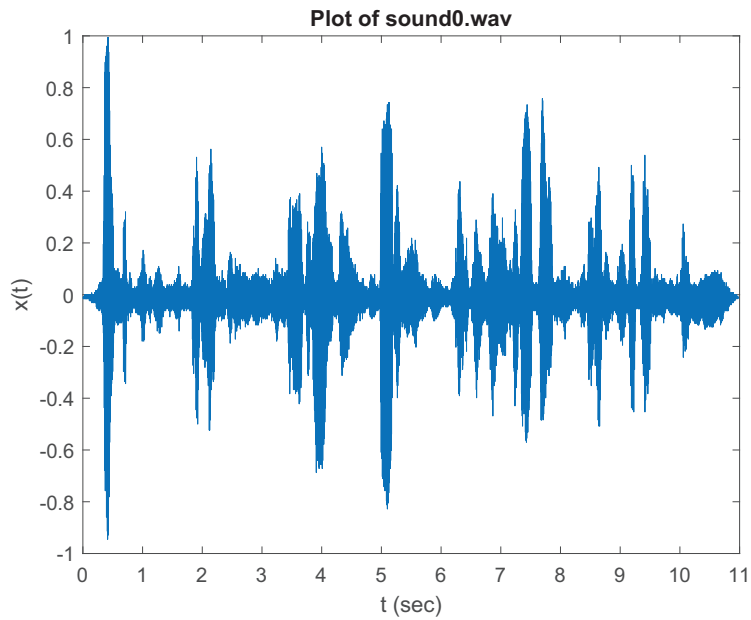


Figure 1.3: Plot of a wave file

```
% p1plotwav.m - This script reads, plays and plots an audio file
%
clear all; close all;           %clean up

[x,fs]=audioread('sound0.wav'); %read the wav file and sampling rate
sound(x,fs)                    %play the sound file

T=1/fs;                        %time between samples
time=0:T:(length(x)-1)*T;      %generate the sampling times
plot(time,x)                   %plot x(t)
axis([0,11,-1,1])
xlabel('t (sec)')
ylabel('x(t)')
title(['Plot of sound0.wav'])
```

which produces Fig. 1.3

Exercises: These practice exercises are for your benefit; they do not need to be turned in.

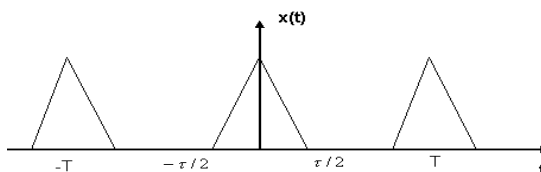
1. **Project I, Chapter 1:** Suppose $x(t)$ is periodic with period T and

$$x(t) = 1 - 2|t|/T \text{ for } |t| \leq T/2.$$

For $T = 1$, plot the Fourier Series of $x(t)$ from $-0.5 \leq t \leq 0.5$, using only the terms $-10 \leq n \leq 10$. Label your plot.

2. **Project I, Chapter 1:** Suppose $x(t)$ is periodic with period T and

$$x(t) = \begin{cases} 1 - 2|t|/\tau, & |t| \leq \tau/2 \\ 0, & \tau/2 < |t| \leq T/2 \end{cases}$$



(a) For $\tau = 1/2$ and $T = 1$, find the Fourier series of $x(t)$.

(b) Now use MATLAB to plot the Fourier Series of $x(t)$ for $-1 \leq t \leq 1$ and truncate the series by omitting all terms except $-10 \leq n \leq 10$. Label your plot.

Turn in your analysis, your code and your plots.

3. **Project 1, Chapter 1:** Consider the periodic half-wave rectified cosine signal in Problem 1:

$$x(t) = [\cos(\pi t)]^+$$

where $[x]^+ = x$ if $x > 0$ and $[x]^+ = 0$ otherwise. Use MATLAB to plot one period of this signal, along with the Fourier Series approximation using only the terms $-6 \leq n \leq 6$. Label your plot. Please turn in both the plot and your code.

4. **Project 1, Chapter 1:** Consider the periodic full-wave rectified cosine signal:

$$x(t) = |\cos(2\pi t)|.$$

(a) Sketch the signal. What is the period? (Careful!)

(b) Find the Fourier Series in complex exponential form.

(c) Use MATLAB to plot one period of this signal, along with the Fourier Series using only the terms $-3 \leq n \leq 3$. Label your plot.

Turn in your code and your plot.

5. **Project 1, Chapter 1:** Consider the periodic full-wave rectified sine signal:

$$x(t) = |\sin(\pi t)|.$$

- (a) What is the period? (Careful! Sketching the signal might help.)
- (b) Find the Fourier Series in complex exponential form.
- (c) Use MATLAB to plot one period of this signal, along with the Fourier Series using only the terms $-3 \leq n \leq 3$. Label your plot. Turn in your analysis, the code and the plot.

Chapter 2

Digital Signal Processing

2.1 The Discrete Fourier Transform

In principle, the frequency content of a continuous-time signal $x(t)$ can be analyzed using the continuous-time Fourier Transform (CFT)

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-2\pi j f t} dt .$$

In practice, however, analysis of real signals like speech or music must be based on a limited number of samples taken over a finite window of time:

$$x[n] = x(nT) , \quad n = 0, \dots, N-1 .$$

How can we analyze the frequency content of $x[n]$, and how is this related to the CFT of the original signal $x(t)$?

Digital signals like $x[n]$ are usually analyzed with the *discrete-time Fourier Transform (DFT)*:

$$\begin{aligned} \text{DFT: } X[k] &= \sum_{n=0}^{N-1} x[n] e^{-2\pi j k n / N} , \quad k = 0, \dots, N-1 , \\ \text{IDFT: } x[n] &= \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{2\pi j k n / N} , \quad n = 0, \dots, N-1 , \end{aligned}$$

Both of these equations are periodic: $x[n] = x[n + N]$ and $X[k] = X[k + N]$.

The DFT has properties very similar to the continuous-time transform $X(f)$. For example, circular convolution in the time domain corresponds to multiplication in the frequency

Property	Time Domain	Frequency Domain
Notation	$x[n], y[n]$	$X[k], Y[k]$
Periodicity	$x[n] = x[n + N]$	$X[k] = X[k + N]$
Linearity	$ax[n] + by[n]$	$aX[k] + bY[k]$
Time Reversal	$x[N - n]$	$X[N - k]$
Circular Time Shift	$x[n - l]$	$X[k]e^{-2\pi jkl/N}$
Circular Frequency Shift	$x[n]e^{2\pi jln/N}$	$X[k - l]$
Complex Conjugate	$x^*[n]$	$X^*[N - k]$
Circular Convolution	$x[n] * y[n]$	$X[k]Y[k]$
Multiplication	$x[n]y[n]$	$(1/N)X[k] * Y[k]$

Table 2.1: Properties of the DFT.

domain:

$$x[n] * y[n] = \sum_{l=0}^{N-1} x[l]y[n-l] \xrightarrow{\text{DFT}} X[k]Y[k] .$$

Other useful properties of the DFT are summarized in Table 2.1.

2.2 Relationship to Other Transforms

If $x[n] = x(nT)$ is a sequence of samples from a continuous-time signal $x(t)$, there are two ways to relate the DFT to the continuous Fourier Transform of $x(t)$.

Sampled Spectrum of a Periodic Signal: If $x(t)$ is periodic with period $T_0 = NT$, the DFT is closely related to the sampled spectrum of $x(t)$. Since the sampled signal is also periodic with period $T_0 = NT$, it has a Fourier Series

$$x_s(t) = x(t)\text{comb}_T(t) = \sum_{k=-\infty}^{\infty} x_k e^{2\pi jkt/T_0}$$

where

$$\begin{aligned} x_k &= \frac{1}{T_0} \int_0^{T_0} x(t)\text{comb}_T(t) e^{-2\pi jkt/T_0} dt \\ &= \frac{1}{T_0} \sum_{n=-\infty}^{\infty} \int_0^{T_0} x(t)\delta(t - nT) e^{-2\pi jkt/T_0} dt \\ &= \frac{1}{T_0} \sum_{n=0}^{N-1} x(nT) e^{-2\pi jkn/N} = \frac{1}{T_0} X[k] . \end{aligned}$$

It follows that $X[k]$ is just a rescaled version of the Fourier Series coefficients of $x_s(t)$

$$X[k] = T_0 x_k .$$

Since the FS gives the line spectrum of the sampled signal

$$X_s(f) = \sum_{k=-\infty}^{\infty} x_k \delta(f - kf_0) = \frac{1}{T_0} \sum_{k=-\infty}^{\infty} X[k] \delta(f - kf_0) ,$$

it follows that $(1/T_0)X[k]$ gives the frequency content of $x_s(t)$ at frequency $f = kf_0 = k/T_0$. Note that the spectral lines occur at multiples of the fundamental frequency f_0 , rather than the sampling frequency f_s .

Approximation to CFT of Windowed Data: We can also view the DFT as an approximation to the CFT of an aperiodic signal $x(t)$. Suppose that $x(t)$ is windowed

$$w(t) = x(t) \Pi\left(\frac{t - T_0/2}{T_0}\right) = \begin{cases} x(t), & 0 \leq t \leq T_0 \\ 0, & \text{elsewhere} \end{cases}$$

The CFT of the windowed signal can be approximated by

$$W(f) = \int_0^{T_0} x(t) e^{-2\pi j f t} dt \approx \sum_{n=0}^{N-1} x(n\Delta t) e^{-2\pi j f n\Delta t} \Delta t$$

where $\Delta t = T = 1/f_s$. If we sample the windowed spectrum every $\Delta f = 1/T_0 = f_0$ Hz, we obtain

$$W(kf_0) \approx T \sum_{n=0}^{N-1} x[n] e^{-2\pi j k n/N} = TX[k] .$$

It follows that $TX[k]$ is an approximation to the spectrum of the windowed signal $w(t)$.

When the approximation is valid, we can use the DFT in place of the CFT to analyze real data signals. For example, we can approximate the energy and power spectral densities of real signals: The windowed signal $w(t)$ is an energy signal with ESD

$$E_w(f)|_{f=kf_0} = |W(kf_0)|^2 \approx T^2 |X[k]|^2 .$$

If $x(t)$ is a power signal, then it has approximate PSD

$$P_x(f)|_{f=kf_0} = \frac{1}{T_0} |W(kf_0)|^2 \approx \frac{T}{N} |X[k]|^2 .$$

The parameters T_0 and f_s need to be chosen carefully, since otherwise the DFT can differ significantly from the CFT. First of all, windowing in the time domain corresponds to convolving with a sinc in the frequency domain, which causes each frequency component of $x(t)$ to “leak” to adjacent frequencies. This problem, called *leakage*, can be reduced by using a larger window T_0 . Secondly, because the DFT operates on sampled data it is also subject to *aliasing*, in which higher frequency components fold over and become low-frequency components in the sampled spectrum. This problem can be eliminated by

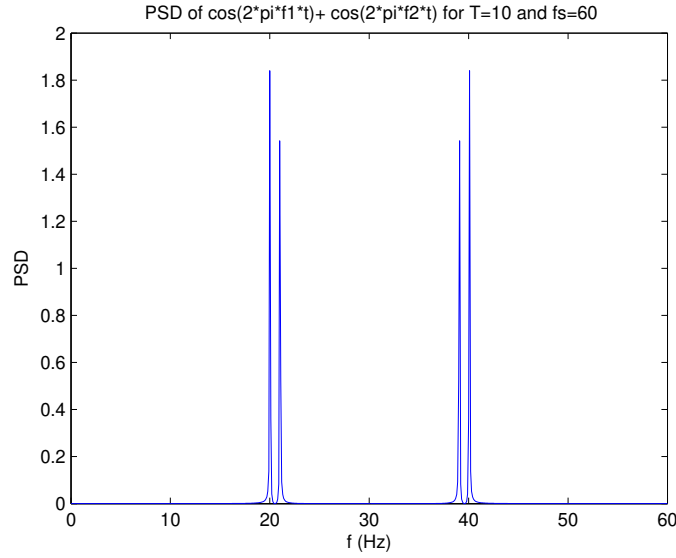


Figure 2.1: PSD of $\cos(2\pi f_1 t) + \cos(2\pi f_2 t)$ for $T = 10$ and $f_s = 60$.

sampling above the Nyquist rate, or by pre-filtering to eliminate the aliased frequencies prior to sampling. Finally, a third type of error called the *picket-fence effect* arises because the DFT cannot resolve distinct frequency components which are closer than $\Delta f = f_0$. This problem can be reduced by using a larger window T_0 .

In summary, the DFT is a good approximation to the CFT when

- $f_s > 2B$, where B is the bandwidth of $x(t)$ (after pre-filtering to remove aliasing)
- T_0 is large enough to provide desired frequency resolution $\Delta f = 1/T_0$ and minimize leakage ($N = T_0/T = T_0 f_s$)

We now use MATLAB illustrate some of these effects with the continuous signal

$$x(t) = \cos(2\pi f_1 t) + \cos(2\pi f_2 t) ,$$

where $f_1 = 20$ Hz and $f_2 = 21$ Hz. This signal has (continuous-time) power spectral density

$$P_x(f) = \frac{1}{4} [\delta(f + f_1) + \delta(f - f_1) + \delta(f + f_2) + \delta(f - f_2)] \quad \text{W/Hz} .$$

The following script (`p1dft.m` on course webpage) approximates this PSD in W/Hz using a DFT with sampling rate $f_s = 60$ and window $T_0 = 10$ s:

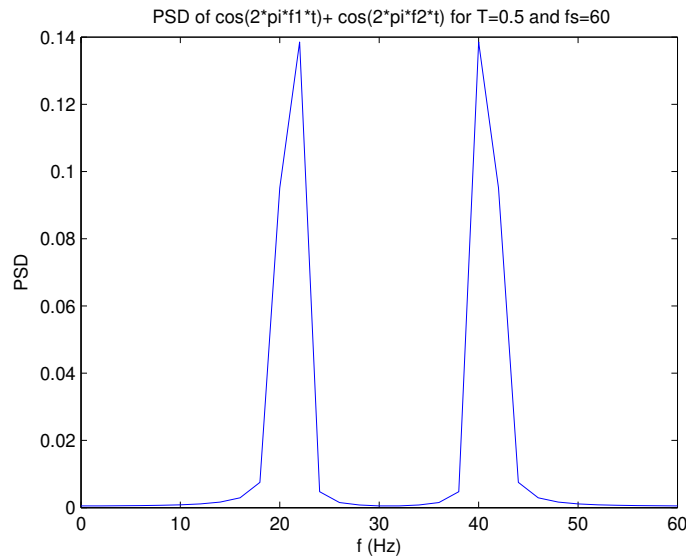


Figure 2.2: PSD of $\cos(2\pi f_1 t) + \cos(2\pi f_2 t)$ for $T = 0.5$ and $f_s = 60$.

```
% pldft - This script plots the power spectral density of a signal composed
% of two tones at frequencies f1 and f2
clear all; close all;

T0=10; fs=60;           %window size and sampling rate
f1=20; f2=21;           %frequencies of the two tones
T=1/fs;                 %time between samples
time=0:T:T0;            %generate sampling times in the window 0 to T0
freq=0:1/T0:fs;          %generate frequencies of the DFT
N=length(time);
x= cos(2*pi*f1*time)+cos(2*pi*f2*time); %sample the signal at rate fs

X=fft(x);
psd=(T/N)*abs(X).^2;     %approximate PSD of the signal
plot(freq,psd)
xlabel('f (Hz)')
ylabel('PSD')
title(['PSD of cos(2*pi*f1*t)+ cos(2*pi*f2*t) for T=', ...
      num2str(T0,2), ' and fs=',int2str(fs)])
```

The resulting plot is shown in Fig. 2.1. While there is clearly some leakage around the frequencies f_1 and f_2 , the script produces a reasonable approximation to the continuous PSD for $|f| \leq f_s/2 = 30$ Hz. If we reduce the window to $T_0 = 0.5$ s, as shown in Fig. 2.2, frequency resolution increases to $\Delta f = 2$ Hz, leakage occurs and it is no longer possible to distinguish the two peaks. On the other hand, if we reduce the sampling rate to $f_s = 30$

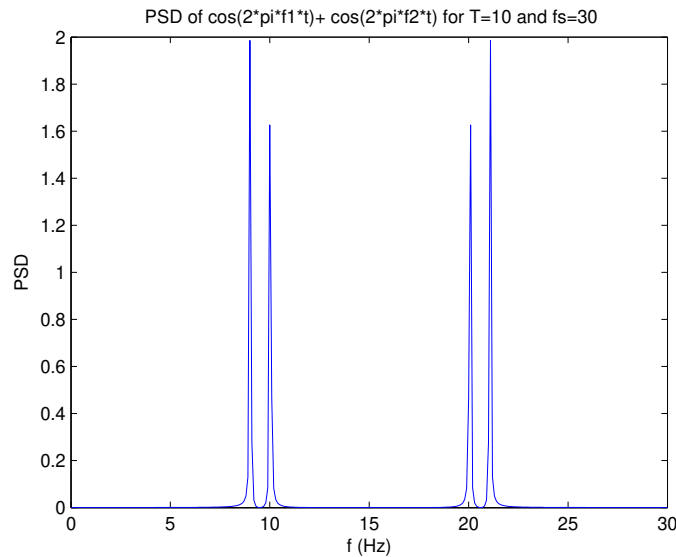


Figure 2.3: PSD of $\cos(2\pi f_1 t) + \cos(2\pi f_2 t)$ for $T = 10$ and $f_s = 30$.

(below the Nyquist rate), then aliasing components appear at $f = 10$ Hz, as seen in Fig. 2.3.

Finally, we also note that MATLAB has a built-in function `periodogram` that can estimate the PSD in an easier way. For example, the script `p1periodogram.m` below

```
% p1periodogram - This script plots the psd of a signal composed of
% two tones at frequencies f1 and f2, using periodogram command
clear all; close all;

T0=10; fs=60;           %window and sampling rate
f1=20; f2=21;          %frequencies of the two tones
T=1/fs;                 %time between samples
time=0:T:T0;            %generate sampling times in window from 0 to T0
N=length(time);
x= cos(2*pi*f1*time)+cos(2*pi*f2*time);    %sample signal at rate fs
periodogram(x,[],'twosided',N,fs)          %plot the periodogram
```

calculates the same PSD as the script `p1dft.m` above. The resulting plot, which is shown in Fig. 2.4, looks a bit different from Fig. 2.1, however, because `periodogram` displays the PSD in dBW/Hz ¹ rather than W/Hz .

We can also use `periodogram` to do power calculations. The command

¹That is to say, `periodogram` plots $10 \log_{10} P_x(f)$ rather than $P_x(f)$.

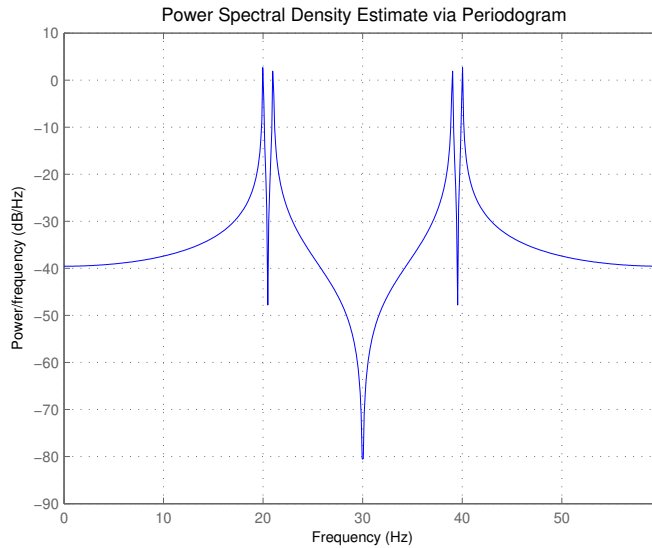


Figure 2.4: PSD in dBW/Hz of $\cos(2\pi f_1 t) + \cos(2\pi f_2 t)$ for $T = 10$ and $f_s = 60$.

```
[psd,freq] = periodogram(x,[],N,fs)
```

returns two vectors: `freq = 0:f0:fs/2` is the vector of frequencies from 0 to $f_s/2$ in increments of f_0 , and `psd` is the (one-sided) power spectral density at those frequencies

```
psd = [P(0), P(-f0)+P(f0), P(-2*f0)+P(2*f0), ... ] .
```

The total signal power can then be approximated by

$$P_x = \int_{-f_s/2}^{f_s/2} P_x(f) df \approx \sum_n P_x(nf_0) f_0 = f_0 * \text{sum}(\text{psd}) .$$

To calculate only the power in a part of the band, say $|f| \leq B$, we can write

$$\int_{-B}^B P_x(f) df \approx \sum_{n=-k}^k P_x(nf_0) f_0 = f_0 * \text{sum}(\text{psd} .* (\text{freq} \leq B)) ,$$

where k is the largest integer such that $kf_0 \leq B$. Here the relational operator `(freq<=B)` zeroes out frequencies outside of $|f| \leq B$, so only power in the desired band is counted.

We conclude this section with an example that illustrates an important claim made in the lecture: Human speech is often said to be approximately bandlimited to 4 kHz. We can test this statement using the tools introduced in this section. The file `sound16.wav` on the

course website contains a brief sample of human speech. Let's write a MATLAB script that performs the following tasks:

- (a) Import the file into MATLAB and find the sampling rate and duration of the clip.
- (b) Plot the PSD of the signal (dBW/Hz vs. Hz) and find the frequency resolution of this plot.
- (c) What is the total power in this signal?
- (d) What fraction of the signal power lies in the band $|f| \leq 4$ kHz?

All four parts of the problem can be done by the following script.

```
% p1powercalc - This script reads in the audio file sound16.wav and calculates
% the PSD and the fraction of signal power contained in |f| <= 4 kHz
clear all; close all;                                %clean up

% part (a)
[x,fs]=audioread('sound16.wav');                     %Read in wav file and sampling rate
N=length(x); T=1/fs; T0=(N-1)*T;                     %file size, sampling time, duration of clip

% part (b)
periodogram(x,[],N,fs);                             %plot PSD of x in dBW/Hz vs Hz
f0=1/T0;                                              %frequency resolution of psd
title('Periodogram of sound16.wav')

% part (c)
[psd,freq]=periodogram(x,[],N,fs);                   %calculate PSD in W/Hz vs Hz
px=f0*sum(psd);                                       %total power of x

% part (d)
px4kHz=f0*sum(psd.*(freq<=4000));                    %power in band -4000 <= f <= 4000
fraction=px4kHz/px;                                   %fraction of power in band
```

- (a) From the script, the sampling rate is $fs=11,025$, the number of samples is $N=37,478$, and the duration is $T0=(N-1)T = 3.3993$ secs.
- (b) The script produces Fig. 2.5 below, which has resolution $f0=0.2942$ Hz.
- (c) As explained above, the total power is $px=f0*\text{sum}(psd)=0.0471$ W.
- (d) The vector $(freq \leq 4000)$ has ones at all frequencies ≤ 4000 and zeros elsewhere, so $px4kHz$ is the power in the band $|f| \leq 4000$, and $\text{fraction}=0.9981$ is the fraction of total power in the band. It follows that $> 99.81\%$ of the signal power is at or below 4 kHz, which strongly supports the claim made in class.

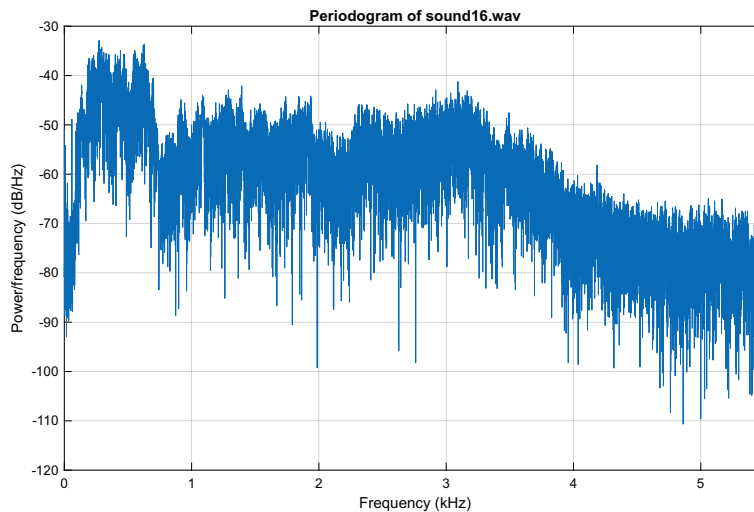


Figure 2.5: PSD of sound16.wav.

Exercises:

1. **Project 1, Chapter 2:** Download `whistle.wav` from the course webpage. This file has been corrupted by an annoying high-pitched whistle.
 - (a) Plot the power spectral density of the signal (dB/Hz vs. Hz) and determine the approximate frequency `fw` of the whistle. (The whistle appears as a spike in the PSD)
 - (b) A Butterworth lowpass filter can be generated with `[b,a]=butter(n,r)`, where `n` is the order of the filter, `r=2*fc/fs`, `fc` is the cutoff frequency of the filter and `fs` is the sampling rate. Design a Butterworth lowpass filter with the smallest possible order that passes signal frequencies up to 3.5 kHz, but suppresses the whistle by at least 30 dB: Take the cutoff frequency `fc` to be halfway between 3.5 kHz and the whistle frequency. What should `r` be? Find an order `n` that suppresses the whistle frequency by 30 dB. *Hint:* Plot the frequency response of the filter for several values of `n`, and find the smallest `n` with a response of -30 dB at the normalized whistle frequency `rw=2*fw/fs`.)
 - (c) Pass the signal through the filter you designed and plot the PSD. Are the higher frequencies suppressed?

Turn in your codes, graphs and the analysis.

2. **Project I, Chapter:** Download `xnoisy.wav` from the course webpage. This file consists of a speech signal corrupted by high-frequency noise.

- (a) Use MATLAB to plot the PSD of the signal (dBW/Hz vs. Hz) and determine the approximate frequency of the noise. (The noise appears as a large spike in the PSD.)
- (b) The approximate bandwidth for voice signals is from 300 Hz to 3.4 KHz. Design an FIR bandpass filter with the bandwidth from 300Hz to 3.4 kHz (cutoff frequencies) using an order of 40. Plot the frequency response of this filter. **NOTE:** Designing a bandpass FIR filter with normalized frequencies r1 and r2: `bw = [r1 r2]; b = fir1(order, bw);`
- (c) For the filter you designed above, how many dB is the high frequency noise suppressed?
- (d) Pass the signal through the filter you designed, renormalize the filtered signal so that $|y[n]| \leq 1$, and write it to a file called `xfiltered.wav`. Plot the PSD of the filtered signal.

For each section, turn in any code or plots generated along with your answers.

3. **Project 1, Chapter 2:** Download `sound7.wav` from the course webpage and import it into MATLAB

- (a) What is the sampling rate? What is the duration of the audio clip?
- (b) Use MATLAB to plot the PSD of the signal (dBW/Hz vs. Hz).
- (c) What is the average power in this signal?
- (d) What fraction of the signal power lies in frequencies $|f| \leq 4$ kHz? *Hint:* Remember that the DFT is periodic, so that $X[-n] = X[N - n]$.

Turn in your codes, plots and the analysis.

4. **Project 1, Chapter 2:** Download `echo.wav` from the course webpage and import it into MATLAB

- (a) What is the sampling rate? What is the duration of the audio clip?
- (b) Use MATLAB to plot the PSD of the signal (dB vs. Hz).
- (c) This audio file consists of a signal corrupted by an echo. If the original signal is $z(t)$, then the corrupted signal is $x(t) = z(t) + (1/2)z(t - T)$ where $T = 1$ sec. Let $x[n]$ and $z[n]$ be the samples of $x(t)$ and $z(t)$. The relationship between the DFTs of $x[n]$ and $z[n]$ can be written in the frequency domain as $X[k] = G[k]Z[k]$. What is $G[k]$?
- (d) Now *design* a filter to eliminate the echo in $x[n]$: Suppose that $Y[k]$ is obtained by filtering $X[k]$, say $Y[k] = H[k]X[k]$. Find a frequency response $H[k]$ that removes the echo (i.e., $Y[k] = Z[k]$). Use MATLAB to apply your filter to `echo.wav`, drop

the imaginary part (`y1=real(y)`), renormalize the filtered signal so that $|y[n]| \leq 1$, and write it to a file called `noecho.wav`. Plot the PSD of the filtered signal. (Hang on to the wave file — I may ask for it later.)

Turn in your codes, plots and the analysis.

5. **Project 1, Chapter 2:** Download `tone.wav` from the homework webpage. This audio file contains a speech clip corrupted by a loud sinusoidal tone.

- (a) Write a MATLAB script that reads the file and plays it. What is the sampling rate? How many samples are there? What is the duration of the clip? What does the clip say? Extra credit: What movie is it from?
- (b) Plot the power spectral density of the signal (dB/Hz vs. Hz) and determine the approximate frequency `ft` of the tone. The tone will be an impulse in the PSD in $0 < f < f_s/2$.
- (c) Now remove the tone: Let $X[k]$ be the DFT of the audio file. A simple way to remove the tone is to zero out the values of $X[k]$ that correspond to it. Because of DFT symmetry, if you zero out $X[k]$ you must also zero out $X[N - k]$. Find `kt`, the value of k that corresponds to the tone frequency `ft`. Write a script that: (1) calculates the DFT $X[k]$, (2) zeros out $X[k]$ and $X[N - k]$ between `kt-200` and `kt+200`, and (3) converts the signal back to the time domain as `y`. *Hints:* `kt` must be an integer. The DFT command is `X=fft(x)` and the inverse DFT is `y = ifft(X)`.
- (d) Play the sound file `y` and plot its PSD to verify the tone is gone. *Hint:* Take the real part of `y` before playing or plotting, to remove small complex errors.

Turn in your codes, graphs and the analysis.

Chapter 3

Sampling and Quantization

In this section, we explore the effects of sampling and quantization on the quality of sound reproduction.

3.1 Sampling Tones

Consider a pure tone of frequency $f_c = 880$ Hz (A above middle C on a piano):

$$x(t) = \sin(2\pi f_c t) \quad , \quad 0 \leq t < T_0 \quad .$$

This is a good approximation to the sound produced by a tuning fork. To generate samples of this signal in MATLAB, we write

```
% pltone.m - This script will generate and play a 5s tone of
% frequency 880 Hz, and write the tone to the file samp4k.wav
fs= 4200;                                %enter sampling frequency
time=0:1/fs:5;                            %generate sampling times
x=sin(2*pi*880*time);                     %generate tone
sound(x,fs);                             %listen to it
audiowrite('samp4200.wav',x,fs);          %write to an external file
```

The file `samp4200.wav` now contains $T_0 = 5$ seconds of the signal $x(t)$ sampled at $f_s = 4.2$ kHz, yielding a total of about $T_0 f_s = 21,000$ samples. Since `audiowrite` defaults to 16-bit quantization, the total file size is 336 kb (42 kB).

According to the sampling theorem, we should be able to do much better. The Nyquist rate of the tone is $2 \times 880 = 1.76$ kHz, so we can reduce f_s without impacting the quality of sound reproduction. Let's try sampling same signal at $f_s = 2.1$ kHz. We could repeat the

process above to generate the new samples, but it is easier to just *downsample* x (take every other sample):

```
% Downsample to fs=2100 and play (above Nyquist)
pause(8)                %wait until last tone has finished
fs2= fs/2;              %reduce sampling rate to 2100
x2=downsample(x,2);      %take every other sample
sound(x2,fs2);           %listen to new file
audiowrite('samp2100.wav',x2,fs2); %write to external file
```

The file `samp2100.wav` also represents 5 seconds of $x(t)$ and sounds exactly the same, but is only half the size of `samp4200.wav` since it is sampled at $f_s = 2.1$ kHz.

Now try sampling at $f_s = 1.4$ kHz by taking every 3rd sample of x

```
% Downsample to fs=1400 and play (*below* Nyquist)
pause(8)                %wait until last tone has finished
fs3= fs/3;              %reduce sampling rate to 1400
x3=downsample(x,3);      %take every third sample
sound(x3,fs3);           %listen to new file
audiowrite('samp1400.wav',x3,fs3); %write to an external file
```

Because we are now sampling below the Nyquist rate, this file sounds very different. Instead of hearing a tone at 880 Hz, you will actually hear an aliased tone at frequency 520 Hz. Can you explain why in terms of the sampling theorem?

3.2 Uniform Quantization

We can also reduce the size of sound files by decreasing the number of bits used to represent each sample in the quantizer. This gives rise to quantization noise, which can also seriously degrade the quality of sound reproduction. In this section, we examine the impact of quantization on the sound quality of simple tones and wave files.

In MATLAB, quantization is performed by the command

```
[index,quantized]=quantiz(signal,partition,levels)
```

Here `signal` is the file we want to quantize, `levels` are the quantizer levels (the amplitudes to which we are rounding the signal) and `partition` gives the boundaries of the quantization intervals, as illustrated in Fig. 3.1. The two outputs of this command are the quantized signal (`quantized`) and a file that indicates which quantization interval each component of `signal`

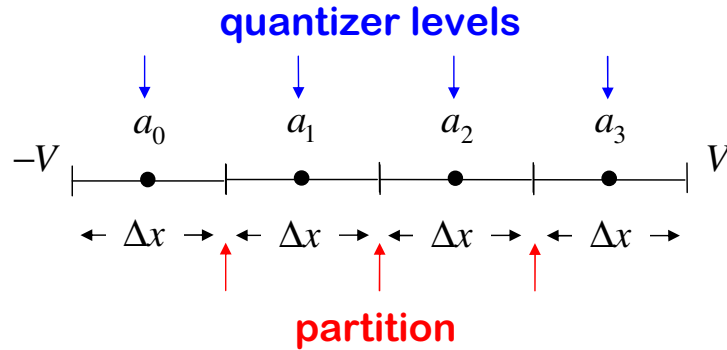


Figure 3.1: Example of $k = 2$ bit uniform quantization.

falls into (index). For example, a $k = 2$ bit uniform quantizer for the interval $-1 \leq x \leq 1$ will have $M = 2^k = 4$ quantizer levels given by `levels=[-0.75, -0.25, 0.25, 0.75]` and the partition will be `partition=[-0.5, 0.0, 0.5]`. Note that the dimension of `partition` is always one less than the dimension of `levels`.

As an example, the following script plots the quantized signal x_q corresponding to each input $-1 \leq x \leq 1$ when $k = 3$ bit uniform quantization is used. A plot of x_q versus x is often called the *quantizer characteristic*.

```
% plquantchar - This script plots the quantizer characteristic for k=3 uniform
% quantization.
V=1;                                %dynamic range
k=3; M=2^k;                          %number bits, levels in quantizer
del=2*V/M;                          %quantizer interval size
partition=-V+del:del:-V+(M-1)*del;   %quantizer boundaries
levels=-V+del/2:del:-V+del/2+(M-1)*del; %quantizer levels
x=-1:0.001:1;                       %generate inputs
[index,xq]=quantiz(x,partition,levels); %calculate quantizer outputs
plot(x,xq)
xlabel('x(t)')
ylabel('[x(t)]_q')
title('Uniform Quantization for k=3')
```

Quantization can significantly degrade the quality of audio files. To illustrate, consider the 880 Hz tone generated in Sec. 3.1. By default, MATLAB will represent the samples of this tone using 16 bits/sample. The following script compares the sound of 16 bits/sample to 8 bits/same 8 bits:

```
% plquantsnr - This script generates and plays a tone of frequency 880 Hz
```

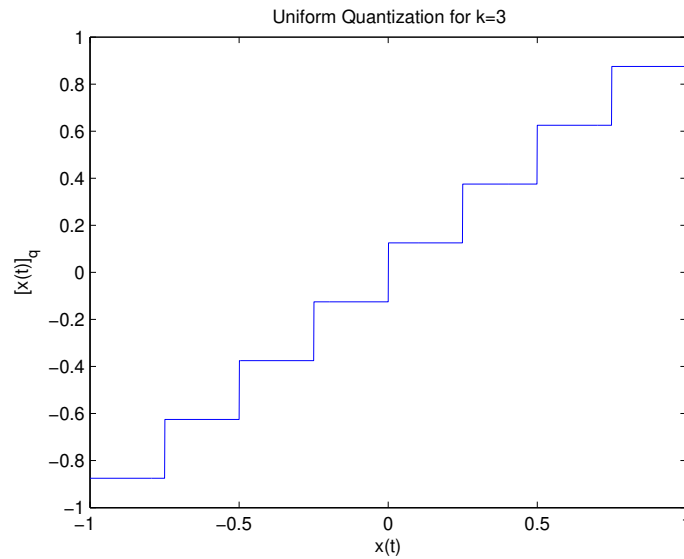


Figure 3.2: Uniform quantizer characteristic for $k = 3$.

```
% and then quantizes the tone using k=8 bit uniform quantization
fs= 4200;                               %enter sampling frequency
time=0:1/fs:5;                           %generate sampling times
x=sin(2*pi*880*time);                   %generate tone
sound(x,fs);                             %listen to it

% Quantize tone and play again
pause(7)                                %wait for last tone to finish
V=1;
k=8; M=2^k;                             %number of bits, levels in quantizer
del=2*V/M;                               %quantizer interval size
partition=-V+del:del:-V+(M-1)*del;       %quantizer boundaries
levels=-V+del/2:del:-V+del/2+(M-1)*del;  %quantizer levels
[index,xq]=quantiz(x,partition,levels);   %quantize signal
sound(xq,fs);                             %listen to it
SNR=10*log10(sum(abs(x).^2)/sum(abs(x-xq).^2)) %calculate actual SNR
```

Do you notice any degradation in sound quality? We can quantify this degradation by calculating the actual signal-to-quantization-noise power ratio (SNR). The power of $x(t)$ can be approximated by

$$P_x = \frac{1}{T_0} \int_0^{T_0} |x(t)|^2 dt \approx \frac{1}{T_0} \sum_{n=0}^{N-1} |x(n\Delta t)|^2 \Delta t = \frac{1}{N} \sum_{n=0}^{N-1} |x[n]|^2.$$

Thus, SNR is given by


```
SNR=10*log10(sum(abs(x).^2)/sum(abs(x-xq).^2))
```

which gives 49.4 dB, which agrees very well with the approximate formula in the notes:

$$(SNR)_{\text{dB}} \approx 10 \log_{10} \left(\frac{P_x}{V^2} \right) + 4.8 + 6.0k = 49.7 \text{ dB} .$$

for $P_x = 1/2$ and $V = 1$. The SNR suggests that the signal power is about 5 orders of magnitude larger than the quantization noise, so the noise is very small.

If we now rerun the script for $k = 7, 6, \dots$, we begin to notice noise in the second tone, which becomes quite noticeable for $k = 5$ ($\text{SNR} \approx 30$ dB). In practice, we normally need to use at least $k \geq 8$ with uniform quantization to avoid a noticeable degradation in audio quality.

3.3 Companding

As discussed in the notes, uniform quantization provides a better SNR for strong signals than for weak signals. For example, the following script repeats the SNR calculation in the last section using the **weaker signal** $x(t) = 0.1 \sin(2\pi 880t)$.

```
% plcompanding - This script compares k=8 bit uniform quantization of a
% tone with quantization using k=8 bit mu-law companding
clear all; close all;
fs= 4200;                               %enter sampling frequency
time=0:1/fs:5;                           %generate sampling times
x=0.1*sin(2*pi*880*time);                 %generate a weak tone
sound(x,fs);                             %listen to it

% quantize tone with k=8 bit uniform quantization and play
pause(7)                                %wait for last time to finish
V=1;
k=8; M=2^k;                             %number of bits, levels in quantizer
del=2*V/M;                              %quantizer interval size
partition=-V+del:del:-V+(M-1)*del;       %quantizer boundaries
levels=-V+del/2:del:-V+del/2+(M-1)*del;   %quantizer levels
[index,xq]=quantiz(x,partition,levels);   %uniform quantization
sound(xq,fs);                            %listen to it
SNR=10*log10(sum(abs(x).^2)/sum(abs(x-xq).^2)) %calculate actual SNR
```

For this weaker signal, SNR drops from 49.4 dB in the last section to just 30.27 dB here, and quantization noise is very noticeable.

If we want to achieve good performance over a wide range of signal powers, we need to perform some type of non-uniform quantization. In long-distance telephony, non-uniform

quantization is often performed by a simple technique called *companding*. In companding, the signal is first compressed by passing it through a non-linear function $c(x)$ called the *compressor*. The compressed signal is then quantized using uniform quantization. The quantized signal is then “decompressed” by passing it through the inverse function of $c(x)$, which is called the *expander*. For example, this final part of script `p1companding.m` performs 8-bit μ -law companding on `x` using $\mu = 255$ and calculates the resulting SNR (see pg. 63 of notes).

```
% quantize tone with k=8 mu-law companding (mu=255) and play
pause(7)
Mu=255;
c=compand(x,Mu,V,'mu/compressor');           %compress
[index,cq]=quantiz(c,partition,levels);      %quantize
xq2=compand(cq,Mu,V,'mu/expander');          %expand
sound(xq2,fs);                               %listen to it
SNRc=10*log10(sum(abs(x).^2)/sum(abs(x-xq2).^2)) %calculate actual SNR
```

In this example, companding increases the SNR to $\text{SNRc}=38.16$ dB and quantization noise is much less noticeable. Thus, companding increases SNR by almost 8 dB, even though the same number of bits are used in the quantizer!

Exercises:

1. **Project I, Chapter 3:** This problem compares the signal-to-quantization noise (SNR) of $k = 5$ bit uniform quantization, with and without companding. First download `sound19.wav` from the course webpage.

- (a) Write a script to import the file into MATLAB and answer the following questions: What is the average power P_x ? What is the smallest V so that $-V \leq x(nT) \leq V$? Use these values to calculate SNR using the **approximate** formula given on pg. 60 of the notes.
- (b) Write a script to quantize the file with $k = 5$ bit uniform quantization, using the value of V from (a). Your file should also calculate the **actual** SNR (in dB) of the resulting file. How does your answer compare to the **approximate** SNR calculated in part (a)?
- (c) Write a script to quantize the file using $k = 5$ bit companding, using the μ -law compressor characteristic on pg. 63 of the notes and $\mu = 20$. What is the **actual** SNR (in dB) of the resulting file? Explain why the SNR is different than in part (b).

(Turn in your code along with the answers)

2. **Project I, Chapter 3:** This problem examines the impact of companding on the signal-to-quantization noise ratio of a sine wave.

- (a) Consider the signal $x(t) = A \cos(80t)$. What is the average power P_x of this signal? For $V = 1$, how should A be chosen so that $10 \log_{10}(P_x/V^2) = d$ dB?
- (b) Write a MATLAB script that generates 10,000 samples of $x(t)$ at a sampling rate of `fs = 500` Hz for $10 \log_{10}(P_x/V^2) = -40, -35, \dots, -5$ dB. For each value of $10 \log_{10}(P_x/V^2)$, quantize the samples with $k = 7$ bit uniform quantization with $V = 1$ and calculate the *actual* signal-to-quantization noise ratio in dB. (Do not use the *approximate* formula given in the notes.) Use MATLAB to plot the SNR versus $10 \log_{10}(P_x/V^2)$. Be sure to label your plot. Turn in the code and the plot.
- (c) Now repeat part (b) using $k = 7$ μ -law companding with compressor characteristic $\mu = 255$ instead of uniform quantization. Plot the *actual* SNR in dB versus $10 \log_{10}(P_x/V^2)$ on the same plot as in part (b). What impact does companding have on the SNR of this signal? When is companding better than uniform quantization? When is it worse? *Hint:* The expander characteristic for μ -law companding is given by

$$e(x) = \text{sgn}(x) \frac{V}{\mu} \left[\exp \left(\frac{|x|}{V} \log(1 + \mu) \right) - 1 \right] .$$

Turn in your code, your plot and your answers.

3. **Project 1, Chapter 3:** This problem examines the impact of undersampling on an audio file.

- (a) Download the file `whistle.wav` from the course webpage and load it into MATLAB with the command `[x,fs,k]=wavread('whistle.wav')`. This file contains a brief audio clip corrupted by a loud whistle. What is the sampling rate `fs` of this file? Play the sound clip and plot the power spectral density. At what frequency (or frequencies) do you observe the whistle?
- (b) Now let's create an **undersampled** version of this wav file with sampling rate `fs1=fs/2` with the command `x1=x(1:2:length(x))`, which takes every other sample of `x`. Play the clip and plot the power spectral density of the new file `x1`. At what frequency is the whistle in file `x1`? How does this compare to the whistle in the original file `x`? Do the whistles sound different? Use sampling theory to explain why a new whistle is present in `x1` and why it occurs at the specific frequency you observed.

Turn in your code, your plot and your answers.

4. **Project 1, Chapter 3:** A good tuning fork produces a pure tone that is very close to a sinusoid, say $x(t) = \cos(2\pi f_1 t)$. In music stores, the most common tuning fork has a frequency of $f_1 = 440$ Hz, which corresponds to the musical note A above middle C.

- (a) Write a MATLAB script that generates 4 seconds of this tone, sampled at $fs = 1000$ Hz, and plays it on the computer soundcard using the `sound` command. Use the `periodogram` command to plot the power spectral density of this sound file. **(Turn in code and plot)**
- (b) Run your script again with the sampling rate changed to $fs = 700$ Hz. Does the sampled tone sound the same? Does the power spectral density look the same? Explain what you see and hear using sampling theory. **(Turn in answers and plot)**

Notes: You will need headphones to hear the tones.